What is WordPiece tokenization?

WordPiece is a tokenization algorithm developed by Google for training the BERT model.

Here, tokenization means breaking a large word or sentence into smaller parts so that the model can understand it more easily.

For example:

`word → w ##o ##r ##d`

Here, `##` shows that this part is not the beginning of the word; it comes later in the word.

Why is it needed?

It is not possible to keep every word in the vocabulary because a language contains an enormous number of words.

So, instead of storing every full word, we store smaller parts of words.

For example:

- playing
- played
- player

The common part here is `play`.

So the model learns these smaller pieces and can later combine them to handle new words as well.

Is WordPiece almost the same as BPE?

Yes, they are quite similar from the training perspective.

Both start with a small vocabulary and then gradually merge smaller parts into larger subwords.

But the big difference is:

- BPE saves the merge rules
- WordPiece saves only the final vocabulary

And during tokenization, WordPiece uses a slightly different logic.

Training algorithm in a simple way

How does it start?

At first, the vocabulary is very small.

It contains:

- special tokens
- alphabet / characters
- `##` before characters that appear inside a word

For example, `word` is initially split like this:

```
w ##o ##r ##d
```

Meaning:

- `w` = the beginning of the word
- `##o`, `##r`, `##d` = the later parts of the word

What happens next?

Then the algorithm checks which two tokens should be merged.

BPE usually merges the pair that appears most frequently.

But WordPiece does not do that.

WordPiece selects a pair using this formula:

$$score = \frac{freq\_of\_pair}{freq\_of\_first \times freq\_of\_second}$$

In simple words, this means:

- it looks at how often a pair appears together
- and also how often each part appears individually

Why does it use this formula?

Because WordPiece wants to merge pairs that have a strong connection when they appear together.

Suppose:

- `un` appears in many words
- `##able` appears in many words

Then even if `un + ##able` appears frequently, both parts are also very common separately.

So WordPiece may think:

“These parts are already useful on their own, so merging them is not that necessary.”

On the other hand:

- `hu`
- `##gging`

These may not be very common individually, but if they often appear together in `hugging`, the algorithm will want to merge them faster.

Let’s look at the example simply

Suppose the vocabulary contains these words and frequencies:

- `"hug"` → 10
- `"pug"` → 5
- `"pun"` → 12
- `"bun"` → 4
- `"hugs"` → 5

Initially, they will be split like this:

- `"hug"` → `h ##u ##g`
- `"pug"` → `p ##u ##g`
- `"pun"` → `p ##u ##n`
- `"bun"` → `b ##u ##n`
- `"hugs"` → `h ##u ##g ##s`

So the initial vocabulary will be:

- `b`

- h
- p
- ##g
- ##n
- ##s
- ##u

Why was the first merge ("##g", "##s")?

It was noticed that ##u appears almost everywhere.

So the pairs containing ##u do not get very impressive scores.

But the pair ##g + ##s is considered relatively more useful.

So the first merge is:

("##g", "##s") → "##gs"

Now a new token is added to the vocabulary:

##gs

And the corpus becomes:

- h ##u ##g
- p ##u ##g
- p ##u ##n
- b ##u ##n
- h ##u ##gs

What happened after that?

Later, this merge happens:

("h", "##u") → "hu"

So now:

- "hug" → hu ##g
- "hugs" → hu ##gs

Then, based on a good score, another merge happens:

`("hu", "##g") → "hug"`

So now:

- `"hug"` → `hug`
- `"hugs"` → `hu ##gs`

This merging process continues until the desired vocabulary size is reached.

What is the main idea?

During training, WordPiece gradually merges smaller parts into meaningful subwords.

How does the tokenization algorithm work?

This is the most important part of WordPiece.

WordPiece does not remember the merge rules.

It only uses the final vocabulary.

Then, when tokenizing a new word, it looks for the biggest subword that exists in the vocabulary.

This is basically called **longest match first**.

Example: `hugs`

Suppose the final vocabulary contains:

- `hug`
- `##s`

Then `hugs` will be tokenized like this:

First, starting from the beginning, we look for the longest matching subword.

We find `hug`.

So the split becomes:

`hugs → hug + ##s`

Final result:

```
["hug", "##s"]
```

What would happen in BPE?

BPE would apply the merge rules in order.

Then the result might be:

```
["hu", "##gs"]
```

So WordPiece and BPE can tokenize the same word differently.

Example: `bugs`

Suppose the vocabulary contains:

- `b`
- `##u`
- `##gs`

Then for:

```
bugs
```

First, the longest match from the beginning is `b`

Remaining part → `##ugs`

Now the longest match from the beginning of `##ugs` is `##u`

Remaining part → `##gs`

`##gs` is in the vocabulary.

So the final tokenization is:

```
["b", "##u", "##gs"]
```

What happens with an unknown word?

This is very important.

If at any stage no matching subword can be found in the vocabulary, then the whole word becomes `[UNK]`.

For example:

`mug`

Suppose the vocabulary does not contain `m` or `##m`

Then the result is:

```
["[UNK]"]
```

`bum`

Suppose `b` exists, and `##u` exists, but `##m` does not exist

Even then, the result will be:

```
["[UNK]"]
```

WordPiece does **not** say:

```
["b", "##u", "[UNK]"]
```

Instead, it treats the entire word as unknown.

How is this different from BPE?

BPE can sometimes keep the known parts of a word and mark only the unknown part as unknown.

But WordPiece is stricter.

If it cannot finish tokenizing the whole word using valid subwords, then:

```
whole word = [UNK]
```